

Dossier GeoScan

Un guide pas-à-pas pour construire, en Laravel, un petit outil de renseignement réseau qui scrape des pages publiques de `shodan.io` et conserve un historique des fiches consultées. Objectif pédagogique : maîtriser le scraping HTML (requêtes HTTP, parsing DOM), pas l'API Shodan.

0 / 10 étapes

AVANT DE COMMENCER

Il te faut : PHP 8.4+, Composer, l'installateur Laravel, et un compte Shodan (même gratuit — certaines sections d'une fiche hôte sont masquées sans connexion). Pas besoin de serveur MySQL : SQLite suffit.

Suis la doc officielle pour l'installation de Laravel : laravel.com/framework/docs/installation.

RÈGLES DU JEU — SCRAPING RESPONSABLE

On scrape des pages publiques, pas l'API payante : c'est le but pédagogique de ce TP. Ça ne dispense pas des bonnes pratiques : identifie ton scraper avec un User-Agent reconnaissable, consulte `shodan.io/robots.txt` avant d'écrire le moindre sélecteur, espace tes requêtes de quelques secondes, et ne scrape que ce dont tu as besoin pour la démo (quelques recherches, quelques fiches). Ce sont les mêmes réflexes qu'on attend d'un vrai projet professionnel de veille OSINT.

PARTIE 1 — FONDATIONS

01 Créer le compte Shodan et repérer les pages cibles

Crée un compte sur Shodan si ce n'est pas déjà fait. Ouvre dans un navigateur les deux types de pages que l'app va scraper : une page de résultats de recherche (par exemple une requête filtrée par pays) et une fiche hôte individuelle (une page détaillant une IP précise). Repère à l'œil les blocs d'information qu'on veut extraire : sur la page de recherche, le nombre total de résultats et les cinq classements « Top » (villes, ports, organisations, produits, systèmes d'exploitation) ; sur la fiche hôte, les informations générales

(pays, ville, organisation, FAI, ASN), les noms d'hôte, les technologies web détectées et les ports ouverts.

Ouvre ensuite les outils de développement du navigateur (inspecteur DOM) sur ces deux pages pour repérer les balises et classes qui entourent chaque information — c'est la carte dont tu auras besoin à l'étape 6.

Validation — tu peux pointer du doigt, sur chaque page, où se trouve chaque donnée listée ci-dessus.

02 Installer le projet Laravel

Installe un nouveau projet Laravel nommé geoscan en suivant la documentation officielle (lien ci-dessus). Configure la connexion base de données sur SQLite plutôt que MySQL — pas de serveur à faire tourner, un simple fichier suffit. Lance les migrations par défaut pour vérifier que la configuration est correcte, puis démarre le serveur de développement et confirme que la page d'accueil s'affiche dans le navigateur.

Validation — la page d'accueil Laravel par défaut s'affiche sans erreur sur le serveur local.

PRIVATE JOKE Le seul moment de ta carrière où « ça marche chez moi » sera littéralement vrai : c'est du SQLite local.

PARTIE 2 — MODÉLISER LA DONNÉE

03 Concevoir le schéma de base de données

Le point le plus important du TP : les fiches Shodan changent tous les jours, mais on ne les retélécharge jamais automatiquement, seulement quand un utilisateur visite une fiche. Il faut donc distinguer l'entité stable de son historique. Prévois quatre tables : une table des recherches effectuées (requête, nombre total de résultats, date), une table des classements associés à chaque recherche (type de classement — ville / port / organisation / produit / OS —, libellé, compteur), une table des hôtes identifiés par leur IP (une ligne par IP jamais vue plus d'une fois), et une table des instantanés d'hôte : une nouvelle ligne à chaque visite, jamais de mise à jour en place, avec la date de récupération et tous les champs de la fiche à ce moment précis.

Réfléchis aux relations entre ces tables avant de passer à l'étape suivante : une recherche a plusieurs classements, un hôte a plusieurs instantanés.

Validation — tu peux dessiner le schéma des 4 tables et leurs relations sans regarder cette page.

04 Écrire les migrations et les modèles Eloquent

Traduis le schéma de l'étape précédente en migrations Laravel, puis crée les modèles Eloquent correspondants avec leurs relations : une recherche possède plusieurs classements (hasMany), un classement appartient à une recherche (belongsTo), un hôte possède plusieurs instantanés (hasMany), un instantané appartient à un hôte (belongsTo). Pense à stocker les listes (noms d'hôte, domaines, technologies web, ports ouverts) dans des colonnes castées en tableau plutôt que dans des tables séparées — plus simple pour un instantané figé dans le temps.

Validation — les migrations s'exécutent sans erreur et un enregistrement de test peut être créé et relu via chaque modèle.

PARTIE 3 — SCRAPER LA RECHERCHE

05 Lire le robots.txt et poser une politique de crawl

Avant d'écrire le premier appel HTTP, ouvre shodan.io/robots.txt et vérifie si les chemins que tu veux scraper (recherche, fiche hôte) sont autorisés. Documente ce que tu trouves. Mets ensuite en place un petit client HTTP réutilisable (au-dessus de Guzzle, la bibliothèque HTTP déjà incluse dans Laravel) qui envoie systématiquement un en-tête User-Agent identifiable et respecte un délai minimum configurable entre deux requêtes — ce délai doit être réglable depuis le fichier `.env` plutôt qu'écrit en dur.

Validation — tu sais dire, chemin par chemin, ce que le robots.txt autorise ou non, et ton client HTTP applique bien le délai configuré.

PRIVATE JOKE Un scraper poli, c'est un scraper qui frappe et attend qu'on lui ouvre — pas qui défonce la porte.

06 Construire le service d'extraction de la page de recherche

Écris un service dédié qui prend une requête de recherche, récupère la page correspondante via ton client HTTP, puis parcourt le HTML pour en extraire le nombre total de résultats et les cinq classements « Top ». Utilise un analyseur de DOM (par exemple `symfony/dom-crawler` avec des sélecteurs CSS) plutôt que des expressions régulières sur du HTML brut — bien plus robuste face aux

petites variations de structure. Une fois l'extraction fiable, enregistre le résultat : une ligne dans la table des recherches et une ligne par élément de classement.

Astuce de travail : sauvegarde une copie locale d'une vraie page de résultats la première fois que tu la récupères, et développe/teste ton analyseur contre cette copie plutôt que de refaire une requête réseau à chaque essai — plus rapide, et plus respectueux du serveur distant.

Validation — une recherche de test crée bien une ligne « recherche » et cinq groupes de classements cohérents avec ce que tu observes dans le navigateur.

07 Formulaire de recherche et affichage des résultats

Ajoute une route et un contrôleur qui présentent un formulaire de recherche simple, déclenchent le service de l'étape précédente à la soumission, puis affichent le total et les cinq classements sous forme de listes ou de petites barres proportionnelles. Pas besoin d'une bibliothèque de graphiques — du Blade et un peu de CSS suffisent amplement pour que ce soit lisible.

Validation — soumettre le formulaire avec une requête réelle affiche des chiffres cohérents avec la même recherche faite manuellement sur shodan.io.

08 Historique des recherches

Ajoute une page qui liste toutes les recherches déjà enregistrées, les plus récentes en premier, avec leur date et leur total de résultats. Cliquer sur une entrée doit réafficher les classements tels qu'ils ont été enregistrés à l'époque — sans jamais relancer une requête vers Shodan. C'est la différence entre « consulter une archive » et « refaire une recherche », et elle doit être nette dans le code comme dans l'interface.

Validation — consulter une ancienne recherche dans l'historique ne déclenche aucune requête HTTP sortante (vérifiable dans les logs ou le débogueur réseau).

PRIVATE JOKE Un historique qui se re-scrape tout seul, c'est plus un espion double qu'une archive.

09 Service de scraping de la fiche hôte, avec cooldown

Écris le second service d'extraction, cette fois pour une fiche hôte identifiée par son IP : pays, ville, organisation, FAI, ASN, noms d'hôte, domaines, technologies web détectées, ports ouverts, et la date de dernière observation par Shodan lui-même. Chaque visite de la fiche dans ton application doit déclencher une tentative de récupération fraîche — mais ajoute un garde-fou : si un instantané pour cette IP a déjà été créé il y a moins de quelques minutes (durée configurable), réutilise-le plutôt que de refaire une requête. C'est ce garde-fou qui transforme « on scrape à chaque clic, sans limite » en « on scrape sobrement, quand c'est utile ».

Validation — visiter deux fois de suite la même fiche ne crée qu'un seul instantané ; visiter après expiration du délai en crée un second.

10 Vue fiche hôte avec ligne du temps

Construis la vue de la fiche hôte : les informations les plus récentes en tête, puis une ligne du temps listant chronologiquement tous les instantanés précédents de cette même IP, avec leur date de récupération. C'est la preuve visuelle que l'application se souvient — un même hôte peut avoir changé d'organisation ou de ports ouverts d'une visite à l'autre, et cette timeline le rend visible.

Pour finir, écris quelques tests qui simulent les réponses HTTP (sans jamais appeler le vrai shodan.io) et vérifient : que l'extraction de la page de recherche produit bien les bons chiffres, que l'extraction de fiche hôte produit bien les bons champs, et que le garde-fou de l'étape 9 fonctionne comme prévu.

Validation — la suite de tests passe entièrement, sans qu'aucune requête réseau réelle ne soit déclenchée pendant son exécution.

PRIVATE JOKE Une IP qui change d'organisation plus souvent que toi de mot de passe, c'est déjà un indice en soi.

Pour aller plus loin : une fois les fondations posées, la connexion à un compte Shodan (variables `SHODAN_EMAIL` / `SHODAN_PASSWORD` dans `.env`, désactivée par défaut) permet de débloquent les sections de fiche hôte réservées aux comptes connectés — à traiter comme une extension, pas comme un prérequis du TP.